

이음_MVP_최종_API_명세서

이음(E-um) MVP 최종 API 명세서

문서 상태: 화면 흐름·DB 구조 재대조 완료 / 해커톤 MVP 구현 기준 최종본

기준 문서: [이음_MVP_최종_화면_흐름.pdf](#), [이음_MVP_최종_DB_구조.pdf](#)

Base URL: [/api/v1](#)

기본 시간대: [Asia/Seoul](#)

데이터 형식: JSON, camelCase

인증: Supabase Access Token

이 문서는 프론트엔드와 FastAPI 백엔드 사이의 계약을 정의한다. 로그인·회원가입·카카오 로그인·로그아웃은 Supabase Auth SDK가 처리하며, 이 문서의 FastAPI 엔드포인트와 구분한다.

0. 최종 범위 요약

영역	최종 결정
FastAPI 외부 API	총 21개: health 1개 + 인증 사용자용 20개
앱 DB	user_profiles , planning_cycles , solar_requests , solar_request_items , solar_messages , tasks , fixed_schedules , plan_blocks , check_ins
인증	이메일·카카오 인증은 Supabase Auth 사용
계획관리	한 개의 /plan-management Route, 서버 screenMode 로 조건부 렌더링
분기 정산	12:00 / 18:00 / 다음 날 04:00 서버 Worker 자동 실행
문서 첨부	제외: PDF 업로드·Document Parse·Information Extract API 없음
반복	제외: 반복 Task·반복 고정 일정 없음
회원탈퇴	실제 삭제 API 없음. 시연용 프론트 처리 후 Supabase signOut()
비밀번호 변경	실제 API 없음. 시연용 프론트 검증만 수행
Task 직접 수정	상세 수정·남은 시간 전용 API 없음. 계획관리 SOLAR 채팅으로만 수정

1. 공통 규칙

1-1. 인증과 사용자 식별

- [GET /health](#) 를 제외한 모든 FastAPI 요청은 [Authorization: Bearer <Supabase access token>](#) 헤더가 필요하다.
- 클라이언트는 [userId](#) 를 요청 본문이나 쿼리에 보내지 않는다.
- 서버는 JWT의 [sub](#) 를 [user_profiles.id](#) 및 각 테이블의 [user_id](#) 로 사용한다.
- 타인의 [requestId](#), [planBlockId](#), [checkInId](#), [taskId](#) 를 보낸 경우 리소스 존재 여부를 노출하지 않고 [404](#) 를 반환한다.

1-2. 공통 헤더

헤더	필수	설명
Authorization	인증 API 필수	Bearer <Supabase access token>
Content-Type	본문이 있는 요청 필수	application/json
X-Request-Id	선택	클라이언트·서버 로그 추적용 ID

별도의 실행용 [Idempotency-Key](#) 는 사용하지 않는다. 실행 중복은 [requestId](#) 와 [solar_requests.status](#) 의 원자적 상태 전이로 막는다. 채팅·결정 버튼 중복은 요청 본문의 [clientEventId](#) 로 막는다.

1-3. 날짜·시각·분기

- 날짜는 [YYYY-MM-DD](#) 형식을 사용한다.

- 시각은 RFC 3339 형식과 UTC offset을 사용한다. 예: `2026-07-31T23:59:59+09:00`.
- 사용자가 마감 날짜만 입력하면 서버는 해당 날짜 `23:59:59+09:00` 으로 저장한다.
- 모든 DB `TIMESTAMPZ` 는 절대 시각으로 저장하고, 논리 날짜와 분기 계산만 `Asia/Seoul` 기준으로 수행한다.

```
MORNING    04:00:00 ~ 11:59:59
AFTERNOON  12:00:00 ~ 17:59:59
EVENING    18:00:00 ~ 다음 날 03:59:59

2026-07-29 02:00
→ logicalDate = 2026-07-28
→ period = EVENING
```

1-4. 공통 성공 응답

```
{
  "data": {}
}
```

`204 No Content` 를 사용하는 API는 응답 본문이 없다.

1-5. 공통 오류 응답

```
{
  "error": {
    "code": "INVALID_REQUEST_STATE",
    "message": "현재 단계에서는 이 작업을 할 수 없어요.",
    "details": {
      "currentStatus": "EXECUTING"
    },
    "traceId": "01J..."
  }
}
```

- `code` 는 프론트 분기 처리용 고정 문자열이다.
- `message` 는 화면에 표시 가능한 사용자용 문구다.
- `details` 는 오류별 보조 정보이며 없으면 빈 객체다.
- `traceId` 는 서버 로그 추적용이다.
- 네트워크 실패는 위 JSON을 받지 못한 통신 실패이며, DB 상태를 실패로 변경하지 않는다.

1-6. 화면용 상태와 DB Enum 구분

다음 값은 프론트 화면 모드이며 DB Enum이 아니다.

```
NEW_CYCLE_ENTRY
ACTIVE_CYCLE_ENTRY
EXECUTION_SUCCESS
EXECUTION_FAILED
NO_ACTIVE_CYCLE
NO_PLANS
IN_PROGRESS
FINALIZING
CHECK_IN_RESULT
DEADLINE_WARNING
```

계획관리의 `COLLECTING`, `CHANGE_CONFIRMATION`, `CHANGE_INPUT`, `FINAL_REVIEW`, `EXECUTING` 은 DB 상태와 화면 모드 이름이 동일하다.

2. FastAPI 엔드포인트 전체 목록

#	Method	Path	역할
1	GET	/health	서버 기본 상태 확인
2	GET	/me	내 프로필 조회
3	PUT	/me/onboarding	닉네임 설정 및 온보딩 완료
4	PATCH	/me/profile	닉네임 수정
5	GET	/bootstrap	앱 실행·복귀 시 전체 상태 동기화
6	GET	/plan-management/state	계획관리 화면 상태 복원
7	POST	/solar/requests	새 SOLAR 요청 생성 및 최초 입력 분석
8	GET	/solar/requests/{requestId}	특정 SOLAR 요청 상세 조회
9	DELETE	/solar/requests/{requestId}	작성 중 또는 실패 요청 취소
10	POST	/solar/requests/{requestId}/messages	질문 답변·추가·수정·삭제 입력
11	POST	/solar/requests/{requestId}/decisions	수정·추가 여부 네/아니요 처리
12	POST	/solar/requests/{requestId}/reopen	최종 검토에서 수정 입력으로 복귀
13	POST	/solar/requests/{requestId}/execute	최종 검토 내용을 실제 계획에 반영 시작
14	GET	/solar/requests/{requestId}/execution	실행 진행·성공·실패 상태 조회
15	POST	/solar/requests/{requestId}/retry	실패한 동일 요청 재실행
16	POST	/solar/requests/{requestId}/acknowledge-result	실행 성공 결과 확인 처리
17	GET	/home/current	현재 논리 날짜·분기 홈 상태 조회
18	PATCH	/plan-blocks/{planBlockId}/check-state	현재 분기 PlanBlock 체크·해제
19	POST	/tasks/deadline-warnings/acknowledge	현재 표시 중인 마감 경고 일괄 확인
20	POST	/check-ins/{checkInId}/acknowledge	분기 정산 결과 확인 처리
21	GET	/calendar	날짜 범위 캘린더 조회

3. 공통 응답 모델

3-1. Profile

```
{
  "id": "uuid",
  "email": "user@example.com",
  "nickname": "유림",
  "onboardingCompleted": true,
  "createdAt": "2026-07-29T09:00:00+09:00",
  "updatedAt": "2026-07-29T09:10:00+09:00"
}
```

email 은 auth.users.email 에서 조회하여 반환하며 user_profiles 에는 중복 저장하지 않는다.

3-2. PlanningCycle

```
{
  "id": "uuid",
  "startDate": "2026-07-29",
  "endDate": "2026-08-04",
  "status": "ACTIVE",
  "activatedAt": "2026-07-29T14:00:00+09:00",
}
```

```

    "endedAt": null
  }

```

3-3. SolarMessage

```

{
  "id": "uuid",
  "sequenceNo": 4,
  "role": "ASSISTANT",
  "kind": "QUESTION",
  "content": "자료구조 과제는 얼마나 걸릴 것 같나요?",
  "metadata": {
    "itemId": "uuid",
    "field": "estimatedMinutes"
  },
  "createdAt": "2026-07-29T14:10:00+09:00"
}

```

`kind` 는 `TEXT` , `QUESTION` , `ERROR` , `DECISION` 만 사용한다.

3-4. SolarRequestItem

```

{
  "id": "uuid",
  "itemOrder": 1,
  "action": "CREATE",
  "actionLabel": "추가",
  "entityType": "TASK",
  "entityLabel": "할 일",
  "status": "INFO_MISSING",
  "statusLabel": "정보 부족",
  "rawLineText": "금요일까지 자료구조 과제",
  "title": "자료구조 과제",
  "summaryText": "정보 부족",
  "normalizedPayload": {
    "title": "자료구조 과제",
    "deadlineAt": "2026-07-31T23:59:59+09:00",
    "estimatedMinutes": null,
    "estimatedMinutesSource": null,
    "remainingMinutes": null,
    "amountText": null,
    "amountSource": null
  },
  "missingFields": ["estimatedMinutes", "amount"],
  "pendingQuestion": {
    "field": "estimatedMinutes",
    "message": "자료구조 과제는 얼마나 걸릴 것 같나요?",
    "attemptCount": 1
  },
  "targetEntityId": null,
  "targetSnapshot": null
}

```

- `action` : `CREATE` , `UPDATE` , `DELETE` .

- `entityType`: `TASK`, `FIXED_SCHEDULE`.
- `status`: `INFO_MISSING`, `READY`, `EXECUTED`.
- `targetEntityId`는 기존 Task·고정 일정 수정 또는 삭제 카드에서만 사용한다.
- `summaryText`, `actionLabel`, `entityLabel`, `statusLabel`은 프론트 표시 편의를 위해 서버가 만든다.

3-5. Task normalizedPayload

```
{
  "title": "자료구조 과제",
  "deadlineAt": "2026-07-31T23:59:59+09:00",
  "estimatedMinutes": 180,
  "estimatedMinutesSource": "USER",
  "remainingMinutes": 180,
  "amountText": "문제 5개",
  "amountSource": "USER"
}
```

- `deadlineAt`을 끝까지 모르면 `null`을 허용한다.
- `estimatedMinutesSource`: `USER` 또는 `AI_ESTIMATED`.
- `amountSource`: `USER`, `AI_ESTIMATED`, `UNKNOWN`.
- `amountSource = UNKNOWN`이면 `amountText = null`이다.
- CREATE 실행 시 DB의 `initial_minutes`, `estimated_minutes`, `remaining_minutes`는 확정된 예상 시간으로 시작한다.
- 기존 Task의 남은 시간 수정은 `remainingMinutes`에 들어가며 별도 Task 수정 API를 만들지 않는다.

3-6. FixedSchedule normalizedPayload

```
{
  "title": "알바",
  "startAt": "2026-08-01T22:00:00+09:00",
  "endAt": "2026-08-02T02:00:00+09:00"
}
```

- 시작 날짜·시각과 종료 날짜·시각이 모두 확정되어야 `READY`가 된다.
- 자정을 넘는 일정도 한 개의 고정 일정으로 저장한다.
- 반복 관련 필드는 존재하지 않는다.

3-7. PlanBlock

```
{
  "id": "uuid",
  "taskId": "uuid",
  "planDate": "2026-07-29",
  "period": "AFTERNOON",
  "allocatedMinutes": 60,
  "allocatedAmountText": "2문제",
  "displayTitle": "자료구조 2문제",
  "displayOrder": 1,
  "status": "PLANNED",
  "checkedAt": null
}
```

홈·CHECK_IN_RESULT·캘린더는 모두 저장된 `displayTitle` 을 그대로 사용한다. 문자열에서 분량이나 시간을 다시 파싱하지 않는다.

3-8. FixedScheduleView

```
{
  "id": "uuid",
  "title": "알바",
  "startAt": "2026-08-01T22:00:00+09:00",
  "endAt": "2026-08-02T02:00:00+09:00",
  "segmentStartAt": "2026-08-01T22:00:00+09:00",
  "segmentEndAt": "2026-08-02T02:00:00+09:00"
}
```

DB에는 원본 일정 한 행만 저장한다. 캘린더 응답의 `segmentStartAt`, `segmentEndAt` 은 선택 날짜·분기와 겹치는 표시 구간을 서버가 계산한 값이다.

3-9. CheckInResult

```
{
  "id": "uuid",
  "checkDate": "2026-07-29",
  "period": "MORNING",
  "totalPlanCount": 3,
  "completedPlanCount": 2,
  "notDonePlanCount": 1,
  "score": 67,
  "replanUnplacedMinutes": 0,
  "finalizedAt": "2026-07-29T12:00:05+09:00",
  "completedPlans": [],
  "notDonePlans": [],
  "cycleEnded": false
}
```

3-10. PlanManagementState

```
{
  "screenMode": "COLLECTING",
  "activeCycle": null,
  "request": {
    "id": "uuid",
    "purpose": "NEW_CYCLE",
    "status": "COLLECTING",
    "rawInput": "금요일까지 자료구조 과제",
    "currentItemOrder": 1,
    "messages": [],
    "requestItems": [],
    "currentQuestion": {
      "itemId": "uuid",
      "field": "estimatedMinutes",
      "message": "얼마나 걸릴 것 같나요?"
    },
    "quickReplies": [
      { "value": "DONT_KNOW", "label": "잘 모르겠어요" }
    ]
  },
  "messages": [
    { "value": "DONT_KNOW", "label": "잘 모르겠어요" }
  ]
}
```

```

    "inputPlaceholder": "예: 2시간 정도 걸려요",
    "pendingItemId": "uuid",
    "decisionPrompt": null,
    "reviewSummary": null,
    "execution": null,
    "createdAt": "2026-07-29T14:00:00+09:00",
    "updatedAt": "2026-07-29T14:03:00+09:00"
  }
}

```

상태에 필요하지 않은 필드는 `null` 또는 빈 배열로 반환한다. 질문 종류별 별도 Route나 별도 상태 Enum은 만들지 않는다.

4. 사용자·온보딩 API

4-1. GET /health

서버가 요청을 받을 수 있는지 확인한다. 인증이 필요 없다.

```

{
  "data": {
    "status": "ok",
    "timestamp": "2026-07-29T14:00:00+09:00"
  }
}

```

Status: 200 OK

4-2. GET /me

설정 화면의 프로필 카드와 개인정보 수정 화면에 필요한 정보를 조회한다.

Request body: 없음

```

{
  "data": {
    "id": "uuid",
    "email": "user@example.com",
    "nickname": "유림",
    "onboardingCompleted": true,
    "createdAt": "2026-07-29T09:00:00+09:00",
    "updatedAt": "2026-07-29T09:10:00+09:00"
  }
}

```

Status: 200 OK

4-3. PUT /me/onboarding

최초 닉네임을 저장하고 온보딩을 완료한다.

```

{
  "nickname": "유림"
}

```

검증: `trim()` 결과가 비어 있으면 422 INVALID_NICKNAME; 중복과 글자 수는 제한하지 않는다.

Response 200: 수정된 Profile 객체

4-4. PATCH /me/profile

개인정보 수정 화면에서 닉네임만 실제로 변경한다.

```
{
  "nickname": "유림이"
}
```

Response 200: 수정된 Profile 객체

이메일은 읽기 전용이다. 새 비밀번호 필드는 프론트에서만 검증하므로 요청 본문에 포함하지 않는다.

5. 앱 초기화·동기화 API

5-1. GET /bootstrap

앱 최초 실행·완전 재실행 때 초기 화면을 결정하고, 포그라운드 복귀 때 전체 캐시를 동기화한다. 이 API는 정산을 시작하거나 DB를 변경하지 않는다.

Request body: 없음

```
{
  "data": {
    "serverTime": "2026-07-29T14:00:00+09:00",
    "profile": {
      "id": "uuid",
      "email": "user@example.com",
      "nickname": "유림",
      "onboardingCompleted": true
    },
    "initialScreen": "IN_PROGRESS",
    "activeCycle": {
      "id": "uuid",
      "startDate": "2026-07-29",
      "endDate": "2026-08-04",
      "status": "ACTIVE",
      "activatedAt": "2026-07-29T10:00:00+09:00",
      "endedAt": null
    },
    "planManagement": {
      "screenMode": "ACTIVE_CYCLE_ENTRY",
      "activeCycle": {},
      "request": null
    },
    "home": {
      "logicalDate": "2026-07-29",
      "period": "AFTERNOON",
      "homeMode": "IN_PROGRESS",
      "blockingNotice": null,
      "activeCycle": {},
      "progress": {
        "checkedCount": 1,
        "totalCount": 3,
        "percentage": 33
      },
      "planBlocks": []
    }
  }
}
```



```
}
}
```

초기 화면 결정 우선순위

1. onboardingCompleted = false
→ NICKNAME_CREATION
2. 현재 SOLAR 요청 존재
→ COLLECTING / CHANGE_CONFIRMATION / CHANGE_INPUT / FINAL_REVIEW /
EXECUTING / EXECUTION_SUCCESS / EXECUTION_FAILED
3. 현재 SOLAR 요청 없음
→ FINALIZING / CHECK_IN_RESULT / DEADLINE_WARNING /
NO_ACTIVE_CYCLE / NO_PLANS / IN_PROGRESS

- 앱 완전 재실행에서는 `initialScreen` 에 따라 최초 라우팅한다.
- 포그라운드 복귀에서는 응답으로 상태와 캐시만 갱신하며 현재 탭을 강제로 바꾸지 않는다.
- 온보딩 전이면 `activeCycle` , `planManagement` , `home` 은 `null` 일 수 있다.

6. 계획관리 조회 API

6-1. GET /plan-management/state

계획관리 탭 포커스 시 현재 요청과 ACTIVE cycle을 조회하여 같은 `/plan-management` Route의 화면을 복원한다.

현재 요청 없음 + ACTIVE cycle 없음

```
{
  "data": {
    "screenMode": "NEW_CYCLE_ENTRY",
    "activeCycle": null,
    "request": null
  }
}
```

현재 요청 없음 + ACTIVE cycle 있음

```
{
  "data": {
    "screenMode": "ACTIVE_CYCLE_ENTRY",
    "activeCycle": {
      "id": "uuid",
      "startDate": "2026-07-29",
      "endDate": "2026-08-04",
      "status": "ACTIVE"
    },
    "request": null
  }
}
```

현재 요청 있음

```
{
  "data": {
    "screenMode": "CHANGE_CONFIRMATION",
```

```

    "activeCycle": null,
    "request": {
      "id": "uuid",
      "purpose": "NEW_CYCLE",
      "status": "CHANGE_CONFIRMATION",
      "messages": [],
      "requestItems": [],
      "currentQuestion": null,
      "quickReplies": [],
      "inputPlaceholder": null,
      "pendingItemId": null,
      "decisionPrompt": {
        "message": "수정하거나 추가할 내용이 있나요?",
        "options": [
          {"value": "YES", "label": "네"},
          {"value": "NO", "label": "아니요"}
        ]
      }
    }
  }
}

```

ACTIVE cycle 수정 요청이면 `activeCycle` 과 `request` 가 동시에 반환된다.

6-2. GET /solar/requests/{requestId}

특정 요청의 메시지·카드·현재 화면 상태를 상세 조회한다. 계획관리 상태 복원이나 개발 중 디버깅에 사용한다.

Path parameter: `requestId` UUID

Response 200 : `PlanManagementState` 와 동일한 구조

타인의 요청 또는 존재하지 않는 요청은 `404 REQUEST_NOT_FOUND` 다.

7. SOLAR 요청 작성 API

7-1. POST /solar/requests

NEW_CYCLE_ENTRY 또는 ACTIVE_CYCLE_ENTRY에서 최초 자연어 입력을 보내 새 요청을 생성하고 SOLAR 분석을 시작한다.

```

{
  "purpose": "NEW_CYCLE",
  "clientEventId": "0190f6c1-...",
  "message": "금요일까지 자료구조 과제
영어 공부 2시간
토요일 22시부터 일요일 2시까지 알바"
}

```

- `purpose` 는 현재 진입 화면에 맞춰 `NEW_CYCLE` 또는 `ACTIVE_CYCLE` 을 보낸다.
- `message` 는 공백이 아닌 여러 줄 텍스트다.
- `clientEventId` 는 사용자 최초 메시지 중복 저장 방지용이다.
- `NEW_CYCLE` 인데 ACTIVE cycle이 있으면 `409 ACTIVE_CYCLE_EXISTS` .
- `ACTIVE_CYCLE` 인데 ACTIVE cycle이 없으면 `409 NO_ACTIVE_CYCLE` .
- 사용자에게 현재 요청이 이미 있으면 `409 ACTIVE_REQUEST_EXISTS` .

```

{
  "data": {
    "screenMode": "COLLECTING",
    "activeCycle": null,
    "request": {
      "id": "uuid",
      "purpose": "NEW_CYCLE",
      "status": "COLLECTING",
      "rawInput": "금요일까지 자료구조 과제...",
      "currentItemOrder": 1,
      "messages": [
        {
          "id": "uuid",
          "sequenceNo": 1,
          "role": "USER",
          "kind": "TEXT",
          "content": "금요일까지 자료구조 과제...",
          "metadata": {},
          "createdAt": "2026-07-29T14:00:00+09:00"
        },
        {
          "id": "uuid",
          "sequenceNo": 2,
          "role": "ASSISTANT",
          "kind": "TEXT",
          "content": "3개의 항목으로 나뉘서 확인하세요.",
          "metadata": {},
          "createdAt": "2026-07-29T14:00:02+09:00"
        },
        {
          "id": "uuid",
          "sequenceNo": 3,
          "role": "ASSISTANT",
          "kind": "QUESTION",
          "content": "자료구조 과제는 얼마나 걸릴 것 같나요?",
          "metadata": {"itemId": "uuid", "field": "estimatedMinutes"},
          "createdAt": "2026-07-29T14:00:02+09:00"
        }
      ]
    },
    "requestItems": [],
    "currentQuestion": {},
    "quickReplies": [],
    "inputPlaceholder": "예: 2시간 정도 걸려요",
    "pendingItemId": "uuid"
  }
}

```

Status: 201 Created

7-2. POST /solar/requests/{requestId}/messages

COLLECTING 또는 CHANGE_INPUT 상태에서 사용자의 질문 답변, 추가·수정·삭제 자연어를 처리한다.

```
{
  "clientEventId": "0190f6c2-...",
  "message": "2시간 정도 걸릴 것 같아"
}
```

Response 200 - 다음 질문이 남은 경우

```
{
  "data": {
    "screenMode": "COLLECTING",
    "activeCycle": null,
    "request": {
      "id": "uuid",
      "status": "COLLECTING",
      "messages": [],
      "requestItems": [],
      "currentQuestion": {
        "itemId": "uuid",
        "field": "amount",
        "message": "분량은 어느 정도인가요?"
      },
      "quickReplies": [
        { "value": "DONT_KNOW", "label": "잘 모르겠어요" }
      ],
      "inputPlaceholder": "예: 문제 5개",
      "pendingItemId": "uuid"
    }
  }
}
```

Response 200 - 모든 카드가 READY가 된 경우

```
{
  "data": {
    "screenMode": "CHANGE_CONFIRMATION",
    "activeCycle": null,
    "request": {
      "id": "uuid",
      "status": "CHANGE_CONFIRMATION",
      "messages": [],
      "requestItems": [],
      "currentQuestion": null,
      "quickReplies": [],
      "inputPlaceholder": null,
      "pendingItemId": null,
      "decisionPrompt": {
        "message": "수정하거나 추가할 내용이 있나요?",
        "options": [
          { "value": "YES", "label": "네" },
          { "value": "NO", "label": "아니요" }
        ]
      }
    }
  }
}
```

```
}
}
```

- 첫 번째 **잘 모르겠어요**에는 SOLAR가 쉬운 표현으로 재질문한다.
- 두 번째에도 예상 시간을 모르면 AI 추정, 분량은 추정 불가 시 **UNKNOWN**, 마감은 **null**로 확정한다.
- 고정 일정 시작·종료 일시는 정확히 확인될 때까지 질문한다.
- 수정·삭제 대상이 애매하면 대상을 임의 선택하지 않고 같은 COLLECTING 화면에서 다시 질문한다.

7-3. POST /solar/requests/{requestId}/decisions

CHANGE_CONFIRMATION 화면의 네/아니오 버튼을 처리한다.

```
{
  "clientEventId": "0190f6c3-...",
  "decision": "YES"
}
```

decision	상태 전이	응답 screenMode
YES	CHANGE_CONFIRMATION → CHANGE_INPUT	CHANGE_INPUT
NO	CHANGE_CONFIRMATION → FINAL_REVIEW	FINAL_REVIEW

Response 200: 갱신된 **PlanManagementState**

7-4. POST /solar/requests/{requestId}/reopen

FINAL_REVIEW에서 [수정]을 눌러 입력 상태로 돌아간다.

Request body: 없음

```
{
  "data": {
    "screenMode": "CHANGE_INPUT",
    "activeCycle": null,
    "request": {
      "id": "uuid",
      "status": "CHANGE_INPUT",
      "messages": [],
      "requestItems": [],
      "inputPlaceholder": "추가하거나 수정할 내용을 입력해 주세요."
    }
  }
}
```

7-5. DELETE /solar/requests/{requestId}

작성 중 나가기 팝업의 [내용 삭제하고 나가기] 또는 EXECUTION_FAILED의 [요청 취소]에 사용한다.

- 삭제 허용 상태: **COLLECTING**, **CHANGE_CONFIRMATION**, **CHANGE_INPUT**, **FINAL_REVIEW**, **FAILED**.
- **EXECUTING**은 삭제·취소할 수 없다.
- **solar_requests**를 물리 삭제하며 **solar_request_items**, **solar_messages**는 CASCADE 삭제한다.
- 삭제가 실패하면 프론트는 현재 화면을 유지한다.

Request body: 없음

Response: **204 No Content**

8. 계획 실행 API

8-1. POST /solar/requests/{requestId}/execute

FINAL_REVIEW의 [모두 등록]을 눌렀을 때 최신 DB 상태를 재검증하고 실행을 시작한다.

Request body: 없음

```
{
  "data": {
    "requestId": "uuid",
    "status": "EXECUTING",
    "screenMode": "EXECUTING",
    "executionStartedAt": "2026-07-29T14:20:00+09:00",
    "executionAttemptCount": 1
  }
}
```

Status: 202 Accepted

실행 전 검증

공통

- 요청 소유자와 requestId 확인
- 현재 status = FINAL_REVIEW 확인
- 중복 실행 여부 확인

NEW_CYCLE

- 실행 시점에도 ACTIVE cycle이 있는지 확인

ACTIVE_CYCLE

- CheckIn 정산 진행 또는 복구 대기 여부 확인
- 대상 cycle이 여전히 ACTIVE인지 확인
- 최신 Task·PlanBlock·고정 일정 기준으로 수정·삭제 대상 재검증

- 검증 성공 시에만 FINAL_REVIEW → EXECUTING 으로 원자적 변경 후 BackgroundTasks를 등록한다.
- 검증 실패 시 status는 FINAL_REVIEW 를 유지하고 실제 실행 작업을 등록하지 않는다.
- 정산 중이면 409 SETTLEMENT_IN_PROGRESS 로 응답하며 정산 완료 후 다시 시도한다.
- ACTIVE cycle이 생겼으면 409 ACTIVE_CYCLE_EXISTS .

8-2. GET /solar/requests/{requestId}/execution

EXECUTING 동안 진행 상태를 확인한다. 네트워크 오류만으로 실패 화면으로 이동하지 않는다.

```
{
  "data": {
    "requestId": "uuid",
    "purpose": "ACTIVE_CYCLE",
    "status": "EXECUTING",
    "screenMode": "EXECUTING",
    "executionStartedAt": "2026-07-29T14:20:00+09:00",
    "executionAttemptCount": 1,
    "executedAt": null,
    "executionResult": null,
    "error": null
  }
}
```

성공 응답

```
{
  "data": {
    "requestId": "uuid",
    "purpose": "NEW_CYCLE",
    "status": "COMPLETED",
    "screenMode": "EXECUTION_SUCCESS",
    "executionStartedAt": "2026-07-29T14:20:00+09:00",
    "executionAttemptCount": 1,
    "executedAt": "2026-07-29T14:20:05+09:00",
    "executionResult": {
      "createdTaskCount": 2,
      "updatedTaskCount": 0,
      "cancelledTaskCount": 0,
      "createdFixedScheduleCount": 1,
      "updatedFixedScheduleCount": 0,
      "deletedFixedScheduleCount": 0,
      "taskCount": 2,
      "fixedScheduleCount": 1
    },
    "error": null
  }
}
```

실패 응답

```
{
  "data": {
    "requestId": "uuid",
    "purpose": "ACTIVE_CYCLE",
    "status": "FAILED",
    "screenMode": "EXECUTION_FAILED",
    "executionStartedAt": "2026-07-29T14:20:00+09:00",
    "executionAttemptCount": 1,
    "executedAt": null,
    "executionResult": null,
    "error": {
      "code": "PLAN_EXECUTION_FAILED",
      "message": "계획을 반영하는 중 문제가 발생했어요.",
      "retryable": true
    }
  }
}
```

서버가 DB에서 실제 **FAILED** 를 반환한 경우에만 EXECUTION_FAILED를 표시한다. 통신 실패 시 프론트는 EXECUTING 화면을 유지하고 같은 API를 [다시 확인]한다.

8-3. POST /solar/requests/{requestId}/retry

FAILED 화면의 [다시 시도]에서 기존 요청과 기존 카드를 재사용하여 다시 실행한다.

Request body: 없음

Response 202 : execute의 EXECUTING 응답과 동일. `executionAttemptCount` 는 1 증가한다.

검증 실패 시 status는 **FAILED** 를 유지하고 실제 작업을 등록하지 않는다.

8-4. POST /solar/requests/{requestId}/acknowledge-result

EXECUTION_SUCCESS의 [홈에서 현재 계획 보기]에서 성공 결과를 확인 처리한다.

Request body: 없음

```
{
  "data": {
    "requestId": "uuid",
    "resultAcknowledgedAt": "2026-07-29T14:21:00+09:00",
    "nextPlanManagementScreenMode": "ACTIVE_CYCLE_ENTRY"
  }
}
```

응답 후 프론트는 홈 탭으로 이동하고 GET /home/current 를 호출한다.

9. 홈 API

9-1. GET /home/current

홈 탭 포커스와 논리 분기 경계 도달 시 현재 상태를 조회한다. 이 API는 정산을 시작하거나 DB를 변경하지 않는다.

공통 응답 골격

```
{
  "data": {
    "serverTime": "2026-07-29T14:30:00+09:00",
    "logicalDate": "2026-07-29",
    "period": "AFTERNOON",
    "homeMode": "IN_PROGRESS",
    "blockingNotice": null,
    "activeCycle": {},
    "progress": null,
    "planBlocks": [],
    "finalizing": null,
    "checkInResult": null
  }
}
```

NO_ACTIVE_CYCLE

```
{
  "data": {
    "logicalDate": "2026-07-29",
    "period": "AFTERNOON",
    "homeMode": "NO_ACTIVE_CYCLE",
    "blockingNotice": null,
    "activeCycle": null,
    "progress": null,
    "planBlocks": [],
    "finalizing": null,
    "checkInResult": null
  }
}
```

NO_PLANS


```
{
  "data": {
    "logicalDate": "2026-07-29",
    "period": "AFTERNOON",
    "homeMode": "NO_PLANS",
    "blockingNotice": null,
    "activeCycle": {},
    "progress": {"checkedCount": 0, "totalCount": 0, "percentage": 0},
    "planBlocks": [],
    "finalizing": null,
    "checkInResult": null
  }
}
```

현재 분기에 고정 일정만 있더라도 홈에는 고정 일정을 표시하지 않으므로 **NO_PLANS** 다.

IN_PROGRESS

```
{
  "data": {
    "logicalDate": "2026-07-29",
    "period": "AFTERNOON",
    "homeMode": "IN_PROGRESS",
    "blockingNotice": null,
    "activeCycle": {},
    "progress": {
      "checkedCount": 1,
      "totalCount": 3,
      "percentage": 33
    },
    "planBlocks": [
      {
        "id": "uuid",
        "taskId": "uuid",
        "planDate": "2026-07-29",
        "period": "AFTERNOON",
        "allocatedMinutes": 60,
        "allocatedAmountText": "2문제",
        "displayTitle": "자료구조 2문제",
        "displayOrder": 1,
        "status": "PLANNED",
        "checkedAt": null
      }
    ],
    "finalizing": null,
    "checkInResult": null
  }
}
```

진행률은 시간 비율이 아니라 **CHECKED PlanBlock 개수 / (PLANNED + CHECKED) 개수** 로 계산한다.

DEADLINE_WARNING blockingNotice

```
{
  "data": {
    "logicalDate": "2026-07-29",
    "period": "AFTERNOON",
    "homeMode": "IN_PROGRESS",
    "blockingNotice": {
      "type": "DEADLINE_WARNING",
      "items": [
        {
          "taskId": "task-id-1",
          "title": "자료구조 과제",
          "deadlineAt": "2026-07-31T23:59:59+09:00",
          "requiredMinutes": 180,
          "availableMinutes": 120,
          "shortageMinutes": 60
        }
      ]
    },
    "activeCycle": {},
    "progress": {},
    "planBlocks": []
  }
}
```

- 경고가 한 개든 여러 개든 `items` 배열 하나만 사용한다.
- `requiredMinutes` 는 현재 분기에서 체크했지만 아직 정산되지 않은 시간을 제외한 effective remaining minutes다.
- 프론트는 `blockingNotice` 가 있으면 기본 홈 위에 차단 화면을 우선 표시한다.

FINALIZING

```
{
  "data": {
    "logicalDate": "2026-07-29",
    "period": "AFTERNOON",
    "homeMode": "FINALIZING",
    "blockingNotice": null,
    "activeCycle": {},
    "progress": null,
    "planBlocks": [],
    "finalizing": {
      "checkInId": "uuid",
      "checkDate": "2026-07-29",
      "period": "MORNING",
      "finalizationStartedAt": "2026-07-29T12:00:00+09:00"
    },
    "checkInResult": null
  }
}
```

`GET /home/current` 통신 실패만으로 정산 실패로 판단하지 않는다. 기존 FINALIZING 화면을 유지하고 같은 API를 [다시 확인]한다.

CHECK_IN_RESULT

```
{
  "data": {
    "logicalDate": "2026-07-29",
    "period": "AFTERNOON",
    "homeMode": "CHECK_IN_RESULT",
    "blockingNotice": null,
    "activeCycle": {},
    "progress": null,
    "planBlocks": [],
    "finalizing": null,
    "checkInResult": {
      "id": "uuid",
      "checkDate": "2026-07-29",
      "period": "MORNING",
      "totalPlanCount": 3,
      "completedPlanCount": 2,
      "notDonePlanCount": 1,
      "score": 67,
      "replanUnplacedMinutes": 0,
      "finalizedAt": "2026-07-29T12:00:05+09:00",
      "cycleEnded": false,
      "completedPlans": [
        {"id": "uuid", "displayTitle": "자료구조 2문제", "status": "COMPLETED"}
      ],
      "notDonePlans": [
        {"id": "uuid", "displayTitle": "영단어 암기", "status": "NOT_DONE"}
      ]
    }
  }
}
```

- 미확인 CheckIn이 여러 개면 `ORDER BY finalized_at DESC, id DESC LIMIT 1` 기준 가장 최근 결과 하나만 반환한다.
- CHECK_IN_RESULT에서는 COMPLETED와 NOT_DONE PlanBlock을 모두 표시한다.
- 캘린더에서는 NOT_DONE 목록을 숨긴다.
- 60점은 피드백 문구 결정에만 사용하며 재계획 여부는 NOT_DONE 존재 여부로 결정한다.
- 프론트는 checkInResult.score를 기준으로 점수별 마스크트 자산을 선택한다.
서버는 마스크트 이름·단계·파일 경로를 별도 필드로 반환하지 않는다.
마스크트 구간은 시각 표현 전용이며 기존 60점 피드백 정책과 NOT_DONE 재계획 정책을 변경하지 않는다.

10. 홈 쓰기 API

10-1. PATCH /plan-blocks/{planBlockId}/check-state

현재 분기 홈에서 PlanBlock을 체크하거나 체크 해제한다.

```
{
  "checked": true
}
```

```
{
  "data": {
    "planBlock": {
```

```

    "id": "uuid",
    "taskId": "uuid",
    "planDate": "2026-07-29",
    "period": "AFTERNOON",
    "displayTitle": "자료구조 2문제",
    "status": "CHECKED",
    "checkedAt": "2026-07-29T14:35:00+09:00"
  },
  "progress": {
    "checkedCount": 2,
    "totalCount": 3,
    "percentage": 67
  }
}
}
}

```

- `checked = true`: `PLANNED` → `CHECKED`.
- `checked = false`: `CHECKED` → `PLANNED`.
- 현재 논리 날짜·현재 분기의 아직 정산되지 않은 PlanBlock만 변경할 수 있다.
- 캘린더에서는 이 API를 호출하지 않는다.

10-2. POST /tasks/deadline-warnings/acknowledge

마감 경고 화면에 표시된 모든 Task를 한 번에 확인 처리한다.

```

{
  "taskIds": ["task-id-1", "task-id-2"]
}

```

```

{
  "data": {
    "acknowledgedTaskIds": ["task-id-1", "task-id-2"],
    "acknowledgedAt": "2026-07-29T14:40:00+09:00"
  }
}

```

- 중복 ID는 서버에서 제거한다.
- 모든 Task가 현재 사용자 소유인지 확인한 뒤 한 트랜잭션에서 저장한다.
- 이미 확인된 Task가 포함되어도 성공으로 처리한다.
- 다른 사용자의 ID 또는 존재하지 않는 ID가 포함되면 전체 요청을 거부한다.
- 응답 후 프론트는 `GET /home/current`를 다시 호출한다.

10-3. POST /check-ins/{checkInId}/acknowledge

현재 화면에 표시된 가장 최근 미확인 CheckIn 결과와 그보다 오래된 미확인 결과를 함께 확인 처리한다.

Request body: 없음

```

{
  "data": {
    "targetCheckInId": "uuid",
    "acknowledgedCheckInIds": ["older-id", "uuid"],
    "acknowledgedCount": 2,
    "resultAcknowledgedAt": "2026-07-29T14:45:00+09:00"
  }
}

```

```
}
}
```

- 대상보다 나중에 생성된 새로운 CheckIn은 확인 처리하지 않는다.
- 같은 요청이 중복 호출되어 대상이 이미 확인된 경우 기존 결과를 반환하고 새 CheckIn을 추가 확인하지 않는다.
- 응답 후 프론트는 `GET /home/current` 를 다시 호출한다.

11. 캘린더 API

11-1. GET /calendar

캘린더 탭의 표시 날짜 범위에 해당하는 PlanBlock, 고정 일정, CheckIn 점수를 조회한다.

Query parameters

이름	필수	형식	설명
from	필수	YYYY-MM-DD	조회 시작 날짜
to	필수	YYYY-MM-DD	조회 종료 날짜

예: `GET /api/v1/calendar?from=2026-07-01&to=2026-08-10`

```
{
  "data": {
    "from": "2026-07-01",
    "to": "2026-08-10",
    "logicalToday": "2026-07-29",
    "currentPeriod": "AFTERNOON",
    "days": [
      {
        "date": "2026-07-29",
        "periods": [
          {
            "period": "MORNING",
            "temporalState": "PAST",
            "checkIn": {
              "id": "uuid",
              "score": 67,
              "totalPlanCount": 3,
              "completedPlanCount": 2,
              "finalizedAt": "2026-07-29T12:00:05+09:00"
            },
            "planBlocks": [
              {
                "id": "uuid",
                "displayTitle": "자료구조 2문제",
                "status": "COMPLETED",
                "displayOrder": 1
              }
            ]
          }
        ],
        "fixedSchedules": []
      }
    ],
    "period": "AFTERNOON",
    "temporalState": "CURRENT",
    "checkIn": null,
    "planBlocks": [

```


사용자 동작	처리 주체	FastAPI 호출
회원탈퇴	시연용 로컬 처리 + Supabase <code>signOut()</code>	<code>DELETE /me</code> 없음
작성 중 나가기 - 계속 작성	팝업 닫기	없음
탭 이동·시스템 뒤로 가기	프론트 내비게이션	필요 시 상태 조회만

13. 오류 코드

HTTP	code	대표 상황	화면 처리
400	MALFORMED_REQUEST	JSON 형식 오류, 필수 본문 누락	일반 오류
401	AUTH_REQUIRED	토큰 없음·만료	로그인 화면 이동
404	PROFILE_NOT_FOUND	앱 프로필 없음	일반 오류
404	REQUEST_NOT_FOUND	요청 없음 또는 타인 요청	현재 화면 유지
404	PLAN_BLOCK_NOT_FOUND	PlanBlock 없음 또는 타인 블록	체크 상태 원상 복구
404	CHECK_IN_NOT_FOUND	CheckIn 없음 또는 타인 결과	홈 재조회
404	TASK_NOT_FOUND	마감 경고 Task 없음	경고 화면 유지
409	ACTIVE_REQUEST_EXISTS	현재 작성·실행·실패·미확인 성공 요청 존재	기존 요청 복원
409	INVALID_REQUEST_STATE	현재 상태에서 허용되지 않는 API	서버 상태 재조회
409	ACTIVE_CYCLE_EXISTS	ACTIVE cycle 중 NEW_CYCLE 시작·실행	현재 계획 수정 안내
409	NO_ACTIVE_CYCLE	ACTIVE_CYCLE 요청인데 활성 cycle 없음	NEW_CYCLE_ENTRY 재조회
409	CYCLE_NOT_ACTIVE	수정 대상 cycle이 이미 종료됨	요청 수정 또는 취소
409	SETTLEMENT_IN_PROGRESS	CheckIn 정산 중 ACTIVE_CYCLE 실행 시도	정산 완료 후 재시도
409	INFORMATION_STILL_MISSING	INFO_MISSING 카드가 있는데 실행 시도	COLLECTING 복원
409	BLOCK_NOT_IN_CURRENT_PERIOD	과거·미래 PlanBlock 체크 시도	기존 상태 유지
409	PERIOD_ALREADY_FINALIZED	정산 완료 후 체크 시도	홈 재조회
409	TARGET_AMBIGUOUS	수정·삭제 대상이 여러 개	COLLECTING에서 재질문
422	INVALID_NICKNAME	닉네임이 비어 있거나 공백만 있음	입력칸 인라인 오류
422	INVALID_DATE_RANGE	calendar <code>from > to</code>	캘린더 기존 데이터 유지
422	INVALID_CHECK_STATE	<code>checked</code> 가 boolean 아님	체크 상태 원상 복구
422	INVALID_TASK_IDS	마감 경고 taskIds가 빈 배열 또는 잘못된 형식	경고 화면 유지
422	INVALID_FIXED_SCHEDULE_TIME	고정 일정 시작·종료가 유효하지 않음	COLLECTING 재질문
429	RATE_LIMITED	과도한 SOLAR 요청	잠시 후 다시 시도
200*	PLAN_EXECUTION_FAILED	BackgroundTasks 실행 트랜잭션 실패	GET /execution의 data.error, 실행 실패 화면
503	SOLAR_UNAVAILABLE	SOLAR 분석 요청 실패	같은 COLLECTING 화면에서 다시 시도

오류 화면 문구 규칙

일반 오류

정보를 불러오지 못했어요.
잠시 후 다시 시도해 주세요.
[다시 시도]

EXECUTING·FINALIZING 진행 상태 조회 오류
진행 상태를 확인하지 못했어요.

작업은 계속 진행 중일 수 있어요.
[다시 확인]

GET /execution 에서 실제 DB status가 FAILED 인 경우는 HTTP 200 이며 data.error 에 실행 실패 코드가 포함된다. 네트워크 오류나 5xx 조회 실패만으로 DB status를 FAILED 로 바꾸지 않는다.

14. API별 DB 읽기·쓰기

API	READ	WRITE
GET /health	선택적 DB 연결 확인	없음
GET /me	user_profiles , auth.users.email	없음
PUT /me/onboarding	user_profiles	user_profiles.nickname , onboarding_completed
PATCH /me/profile	user_profiles	user_profiles.nickname
GET /bootstrap	프로필, ACTIVE cycle, 현재 request/items/messages, home/checkIn 관련 테이블	없음
GET /plan-management/state	planning_cycles , solar_requests , solar_request_items , solar_messages	없음
GET /solar/requests/{id}	request/items/messages/cycle	없음
POST /solar/requests	현재 request, ACTIVE cycle	solar_requests , solar_messages , solar_request_items
POST /messages	request/items/messages, 대상 tasks/fixed_schedules	solar_messages , solar_request_items , solar_requests
POST /decisions	request/messages	solar_messages , solar_requests
POST /reopen	request	solar_requests , 선택적 solar_messages
DELETE /solar/requests/{id}	request	request 물리 삭제 + 하위 CASCADE
POST /execute	request/items/cycle/checkIn 상태	solar_requests EXECUTING 전이; BackgroundTasks 등록
GET /execution	solar_requests	없음
POST /retry	FAILED request/items/cycle/checkIn 상태	solar_requests EXECUTING 전이; BackgroundTasks 등록
POST /acknowledge-result	COMPLETED request	solar_requests.result_acknowledged_at
GET /home/current	cycle, plan_blocks, tasks, check_ins	없음
PATCH /check-state	plan_block, 현재 분기, check_in	plan_blocks.status , checked_at
POST /tasks/deadline-warnings/acknowledge	tasks	tasks.deadline_warning_acknowledged_at
POST /check-ins/{id}/acknowledge	check_ins	대상 및 이전 check_ins.result_acknowledged_at
GET /calendar	planning_cycles, plan_blocks, check_ins, fixed_schedules	없음
SOLAR 실행 Worker	request/items/cycle/tasks/schedules/blocks	cycle/tasks/fixed_schedules/plan_blocks/items/request 전체 트랜잭션
CheckIn 정산 Worker	cycle/blocks/tasks/schedules/check_ins	check_in 결과, block 상태, task 남은 시간, 미래 blocks, cycle 종료

15. 핵심 시퀀스

15-1. 최초 7일 계획 생성


```

POST /solar/requests (purpose = NEW_CYCLE)
→ COLLECTING

POST /solar/requests/{id}/messages 반복
→ 정보 부족 해결
→ CHANGE_CONFIRMATION

POST /solar/requests/{id}/decisions { decision: NO }
→ FINAL_REVIEW

POST /solar/requests/{id}/execute
→ 202 EXECUTING

GET /solar/requests/{id}/execution 반복
→ COMPLETED + EXECUTION_SUCCESS

POST /solar/requests/{id}/acknowledge-result
→ GET /home/current
→ IN_PROGRESS / NO_PLANS

```

15-2. 현재 7일 계획 수정

```

GET /plan-management/state
→ ACTIVE_CYCLE_ENTRY

POST /solar/requests (purpose = ACTIVE_CYCLE)
→ 카드 생성 및 질문

FINAL_REVIEW → execute
→ 최신 cycle·Task·고정 일정 재검증
→ 현재 미체크 PLANNED·미래 PLANNED 재계획
→ 기존 CHECKED·과거 결과 유지

```

15-3. 실행 실패

```

GET /execution
→ status = FAILED
→ EXECUTION_FAILED

다시 시도
POST /solar/requests/{id}/retry
→ EXECUTING

요청 취소
DELETE /solar/requests/{id}
→ ACTIVE cycle 있으면 ACTIVE_CYCLE_ENTRY
→ 없으면 NEW_CYCLE_ENTRY

```

15-4. 분기 정산과 결과 확인

```

12:00 / 18:00 / 다음 날 04:00
→ 서버 Worker 정산 시작
→ check_ins.finalized_at = NULL
→ GET /home/current = FINALIZING

정산 완료
→ CHECKED → COMPLETED
→ PLANNED → NOT_DONE
→ Task.remaining_minutes 갱신
→ 미래 PlanBlock 재계획
→ check_ins.finalized_at 저장
→ GET /home/current = CHECK_IN_RESULT

POST /check-ins/{id}/acknowledge
→ GET /home/current
→ DEADLINE_WARNING / NO_PLANS / IN_PROGRESS / NO_ACTIVE_CYCLE

```

15-5. 마감 경고

```

GET /home/current
→ blockingNotice.type = DEADLINE_WARNING
→ items 배열 전체 표시

POST /tasks/deadline-warnings/acknowledge
{ taskId: 화면에 표시된 모든 ID }

GET /home/current
→ 원래 홈 상태 표시

```

16. 구현하지 않는 API

제외 API·기능	이유 / 대체 흐름
FastAPI 로그인·회원가입 API	Supabase Auth 사용
POST /logout	Supabase <code>signOut()</code> 사용
DELETE /me	회원탈퇴는 해커톤 시연용 프론트 처리
비밀번호 찾기·변경 API	MVP 미구현
문서 업로드·파싱 API	PDF·Document Parse 범위 제외
반복 일정 API	반복 기능 제외
ACTIVE cycle 강제 종료·교체 API	자연 종료 후 새 cycle 시작
GET /planning-cycles/active	bootstrap·plan-management-home 응답으로 충분
GET /check-ins/latest-result	GET /home/current 가 상세 결과까지 반환
POST /tasks/{taskId}/deadline-warning/acknowledge	일괄 API로 교체
PATCH /tasks/{taskId}/remaining-minutes	SOLAR ACTIVE_CYCLE 요청으로 수정
Task 상세 CRUD API	계획관리 SOLAR 채팅으로 추가·수정·삭제
캘린더 체크 API	캘린더 조회 전용

17. 화면 흐름·DB 구조 더블체크 결과

17-1. 화면 행동 누락 검사

화면 행동	연결 API / 처리	검토
앱 최초 실행·재실행	GET /bootstrap	일치
포그라운드 복귀	GET /bootstrap, 현재 탭 유지	일치
닉네임 생성	PUT /me/onboarding → GET /bootstrap	일치
계획관리 탭 포커스	GET /plan-management/state	일치
최초 입력	POST /solar/requests	일치
질문 답변·추가 수정	POST /messages	일치
네/아니요	POST /decisions	일치
최종 검토 수정	POST /reopen	일치
작성 중 삭제하고 나가기	DELETE /solar/requests/{id}	일치
모두 등록	POST /execute	일치
실행 중 조회	GET /execution	일치
실행 실패 재시도	POST /retry	일치
성공 결과 확인	POST /acknowledge-result	일치
홈 탭 포커스·분기 경계	GET /home/current	일치
PlanBlock 체크·해제	PATCH /check-state	일치
마감 경고 확인	일괄 acknowledge	일치
CheckIn 결과 확인	POST /check-ins/{id}/acknowledge	일치

화면 행동	연결 API / 처리	검토
캘린더 조회	GET /calendar	일치
닉네임 변경	PATCH /me/profile	일치
로그아웃	Supabase signOut, FastAPI 없음	일치
회원탈퇴 시연	프론트 + signOut, DELETE /me 없음	일치

17-2. DB 컬럼 공급 가능 여부

응답 데이터	주요 DB 출처	검토
프로필	user_profiles + auth.users.email	가능
ACTIVE cycle	planning_cycles	가능
계획관리 상태	solar_requests.status/purpose	가능
채팅 복원	solar_messages sequence_no	가능
카드·질문 복원	solar_request_items payload/missing/pending	가능
실행 성공 건수	solar_requests.execution_result	가능
실행 실패	solar_requests.error_code/error_message	가능
홈 실행 문구	plan_blocks.display_title	가능
홈 진행률	현재 PLANNED/CHECKED 개수	가능
마감 경고	tasks + effective remaining 계산	가능
FINALIZING	check_ins.finalized_at IS NULL	가능
CHECK_IN_RESULT	check_ins + plan_blocks.check_in_id	가능
과거 점수	check_ins.score	가능
과거 완료 목록	COMPLETED plan_blocks	가능
미래 최신 계획	현재 남아 있는 PLANNED plan_blocks	가능
자정 초과 고정 일정	fixed_schedules start_at/end_at	가능

17-3. 이전 API 초안에서 제거·수정한 항목

- `DELETE /me` 제거: 실제 회원탈퇴를 구현하지 않는다.
- 문서 업로드·Document Parse API 제거.
- 반복 Task·반복 고정 일정 필드와 API 제거.
- ACTIVE cycle 강제 교체 흐름 제거.
- 홈에서 고정 일정 목록 제거. 고정 일정은 배치 계산과 캘린더에서만 사용한다.
- 마감 경고 단일 Task API 제거 후 `taskIds` 일괄 API로 통일.
- 별도 최근 CheckIn 조회 API 제거 후 `GET /home/current`에 결과 상세 포함.
- 캘린더 `emptyReason` 제거. 세 배열이 비면 통합 빈 화면으로 처리.
- 조회 API가 정산을 시작한다는 규칙 제거. 정산은 Worker 전용이다.
- 실행 중 네트워크 오류를 DB 실패로 간주하지 않도록 명확화.
- PlanBlock 시간 30분 배수 제약 제거. 정수 분 단위 허용.
- 자정을 넘는 고정 일정 지원으로 수정.

18. 4일 남은 MVP 구현 우선순위

우선순위	구현 묶음	완료 기준
P0-1	인증 연동, GET/PUT/PATCH me, bootstrap	로그인 후 올바른 초기 화면 복원
P0-2	plan-management state, request/messages/decisions/reopen/delete	채팅 입력부터 FINAL_REVIEW까지 동작

우선순위	구현 묶음	완료 기준
P0-3	execute/execution/retry/acknowledge-result + 실행 Worker	새 계획 생성·수정과 성공/실패 화면 동작
P0-4	home/current + check-state	현재 분기 목록·체크·진행률 동작
P0-5	CheckIn Worker + acknowledge	분기 정산·결과·재계획 동작
P0-6	calendar + deadline warning acknowledge	과거·현재·미래 표시와 경고 동작
P1	health, 특정 request 상세, 세부 오류 코드 정리	시연 안정성·디버깅 보강

최종 결론: 위 21개 FastAPI 엔드포인트와 Supabase Auth 클라이언트 동작만으로 최종 화면 흐름과 9개 앱 DB 테이블을 빠짐없이 연결할 수 있다. 새로운 핵심 Route, 테이블, 상태 Enum, 문서·반복 기능 API는 추가하지 않는다.